

KUBERNETES IN PRODUCTION · ISSUE 06 OF 08

Services, Ingress and NetworkPolicy.

Most ingress bugs are a label or a port mismatch.

06

AR

Akhilesh Ranjan Singh

Senior DevSecOps Engineer · github.com/iarsingh

Swipe →

Service discovery basics

01 ClusterIP + DNS

`api.payments.svc.cluster.local` resolves to a stable IP.

02 EndpointSlices

The ready pods behind the Service.

03 Headless Service

DNS returns pod IPs directly, used by StatefulSets.

04 Selector

Labels must match the pod template exactly.

The request path

1 DNS

2 Load balancer

3 Ingress controller

4 Service

5 EndpointSlice

6 Pod

Debug in this order. Each hop has one command that proves it works.

Ingress troubleshooting

- ☒ DNS resolves to the LB address
- ☒ TLS secret exists and is valid
- ☒ ingressClassName matches controller
- ☒ Host and path rules match
- ☒ Service selector matches pod labels
- ☒ targetPort matches containerPort
- ☒ Pods are Ready
- ☒ Controller logs are clean

Default deny, then allow

YAML

```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata: { name: default-deny, namespace: payments }
spec:
  podSelector: {}
  policyTypes: [Ingress]
---
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata: { name: allow-ingress-controller, namespace: payments }
spec:
  podSelector: { matchLabels: { app: api } }
  ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: ingress-nginx
```

NetworkPolicy gotchas

DO

- ✓ Start with default-deny per namespace
- ✓ Allow DNS egress explicitly
- ✓ Test with a throwaway pod

AVOID

- ✗ Assume your CNI enforces policy
- ✗ Deny egress and forget kube-dns
- ✗ Allow all from every namespace

KEY TAKEAWAY

Walk the hops in order. Check labels and ports first.

- Six hops from DNS to pod
- Selectors and targetPorts break most often
- Default-deny, then allow

YOUR TURN

Which hop has cost you the most debugging time?

FOLLOW AKHILESH RANJAN SINGH

Next in the series: #07 Secrets management patterns

Save · Repost

